
AGENDA 5

Git e GitHub

Parte 2 - Primeiro push



SEAD- Superintendência de Ensino e Pesquisa nas
modalidades EaD e Aberta

GOVERNO DO ESTADO DE SÃO PAULO
EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

DESENVOLVIMENTO DE SISTEMAS I

Expediente

Autores:

PAULO EDUARDO CARDOSO ANDRADE

TATIANE TOLENTINO DE ASSIS

Revisão Técnica:

Kelly Cristiane de Oliveira Dal Pozzo

São Paulo – SP, 2025



MOMENTO REFLEXÃO



Pense na seguinte situação: você passou horas desenvolvendo um projeto importante no seu computador. Tudo estava funcionando bem, mas de repente o computador apresentou problemas e você perdeu seus arquivos. Ou então, imagine que você precisa mostrar o projeto para alguém, mas está longe da máquina em que salvou tudo.

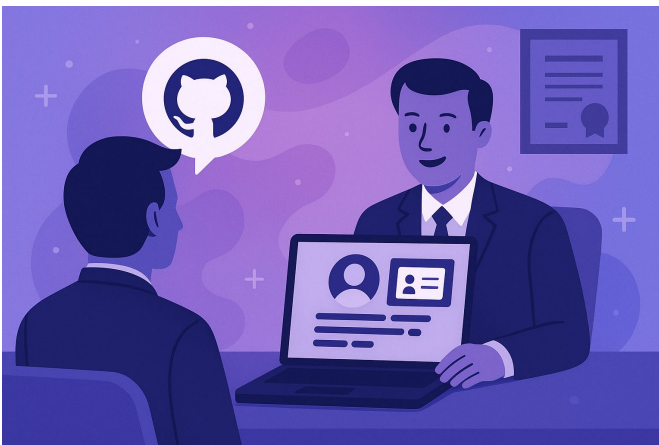
O que você faria nessa situação?

É justamente para evitar esse tipo de problema que usamos ferramentas como o Git e o GitHub. Além de permitir que você não perca seu trabalho, essas ferramentas também possibilitam que você registre cada etapa da evolução do projeto, colabore com outras pessoas e mantenha tudo salvo na nuvem, pronto para ser acessado de qualquer lugar.

O GitHub não é apenas um lugar para guardar código: é uma forma de garantir segurança, organização e colaboração nos seus projetos.



POR QUE APRENDER?



No mercado de tecnologia, não basta apenas saber programar. É necessário organizar, compartilhar e evoluir projetos de forma profissional. Nesse contexto, o Git e o GitHub tornaram-se ferramentas essenciais no dia a dia dos desenvolvedores e, por isso, são frequentemente exigidos em vagas de estágio e emprego na área de tecnologia.

Aprender a utilizar essas ferramentas permite acompanhar a evolução de um projeto, registrar o histórico de alterações realizadas no código e recuperar versões anteriores sempre que necessário. Além disso, o uso do Git e do GitHub

facilita o trabalho colaborativo, permitindo que várias pessoas contribuam para o mesmo projeto de forma organizada e segura.

Outra vantagem importante é a possibilidade de construir um portfólio profissional. Ao publicar projetos no GitHub, o desenvolvedor consegue demonstrar suas habilidades na prática, apresentando exemplos reais de código

e soluções desenvolvidas ao longo do tempo. Muitos recrutadores e empresas utilizam essa plataforma para analisar o perfil de candidatos durante processos seletivos.

Nesse sentido, manter um perfil ativo no GitHub pode funcionar como um verdadeiro currículo interativo para desenvolvedores. Repositórios organizados mostram a capacidade de estruturar projetos, commits bem descritos demonstram disciplina e atenção aos detalhes, e projetos pessoais revelam iniciativa e interesse em aprender. Além disso, contribuições em projetos de outras pessoas evidenciam espírito colaborativo, uma característica bastante valorizada no mercado de tecnologia.

Durante entrevistas de emprego, é comum que recrutadores perguntem se o candidato possui um perfil no GitHub para apresentar seus projetos. Um perfil bem estruturado pode se tornar um diferencial importante entre candidatos com níveis semelhantes de conhecimento técnico.

PARA COMEÇAR O ASSUNTO

Agora que já compreendemos a importância do Git e do GitHub no mercado de trabalho e como essas ferramentas podem se tornar um diferencial em processos seletivos e oportunidades profissionais, chegou o momento de colocarmos esse conhecimento em prática.

O primeiro passo será preparar o nosso ambiente de desenvolvimento. Para isso, iremos instalar e configurar duas ferramentas essenciais: o Git, responsável pelo controle de versão dos projetos, e o Visual Studio Code, um editor de código amplamente utilizado por desenvolvedores no mundo todo.



Com essas ferramentas configuradas, você terá um fluxo de trabalho semelhante ao utilizado em ambientes profissionais de desenvolvimento. A partir desse momento, será possível criar e gerenciar projetos diretamente no seu computador, registrar e organizar cada alteração realizada nos arquivos e conectar seus projetos ao GitHub para publicá-los na internet.

Além de facilitar a organização do código, esse processo também permite atualizar seu portfólio de forma contínua, registrando sua evolução como desenvolvedor ao longo do curso.

MERGULHANDO NO TEMA

Depois de criar e personalizar o seu perfil no GitHub, chegou a hora de dar o próximo passo: usar a plataforma como ferramenta ativa de desenvolvimento e aprendizado. Até aqui, você estruturou a base do seu portfólio digital; agora, vamos evoluir para a prática de publicar projetos e registrar suas experiências. Esse é o momento em que o GitHub deixa de ser apenas um perfil configurado e passa a se tornar um espaço vivo, que mostra seu crescimento como estudante e futuro profissional da área de tecnologia.

Imagine que você está trabalhando em um projeto no seu computador: pode ser uma página web, um script em Python ou um aplicativo simples. Se esse projeto ficar apenas salvo em uma pasta local, ninguém mais terá acesso a ele. Mas, ao conectá-lo ao GitHub, você ganha várias vantagens:

- **Organização:** seus arquivos ficam versionados, com histórico de alterações.
- **Colaboração:** você pode compartilhar com colegas e professores, que podem sugerir melhorias.
- **Acesso remoto:** você pode abrir seus projetos de qualquer lugar, sem depender do computador onde começou.
- **Portfólio:** cada repositório publicado fortalece sua presença profissional online.

Nesta etapa, você vai aprender a transformar seu computador em uma oficina conectada ao GitHub, publicando seus projetos de forma organizada, colaborativa e profissional.

Preparação do ambiente

Antes de começar a postar seus projetos no GitHub, é necessário preparar o ambiente de trabalho no seu computador. Isso significa ter ou instalar duas ferramentas fundamentais, em nossas atividades vamos utilizar:



Visual Studio Code (VS Code): considerado um editor de código moderno, leve e com suporte integrado ao Git. Lembre-se que já realizamos a instalação na agenda 02.



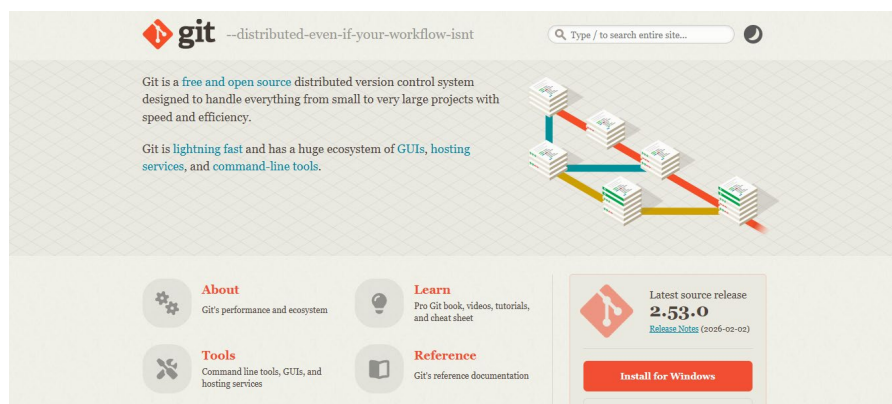
Git: responsável pelo controle de versão e comunicação com o GitHub.

Como a instalação do VSCode já foi realizada na agenda 02, agora vamos instalar o software Git.

Instalação do Git

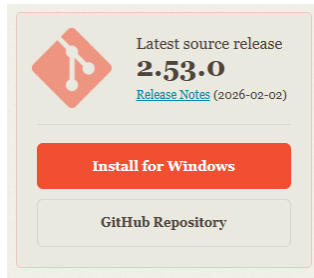
Para iniciar o processo instalação é necessário fazer o download do Git, para isso acesse o site oficial:

<https://git-scm.com>

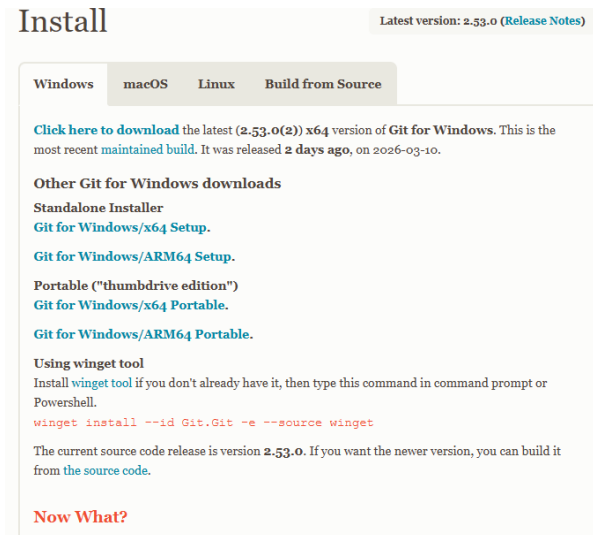


Na área de downloads selecione a versão compatível com o seu sistema operacional (Windows, Mac ou Linux).

1.



2.

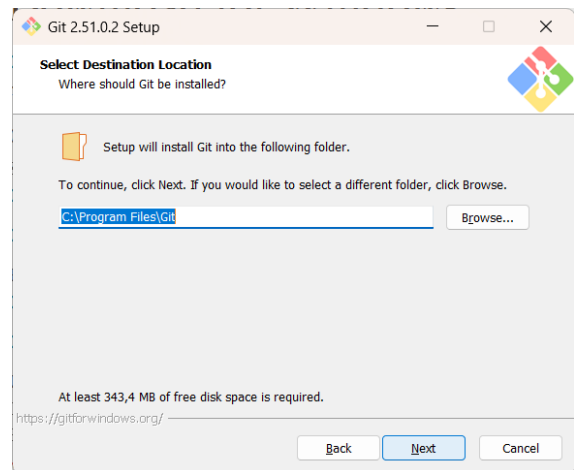


Após download, selecione o arquivo e execute a instalação com as configurações padrão:

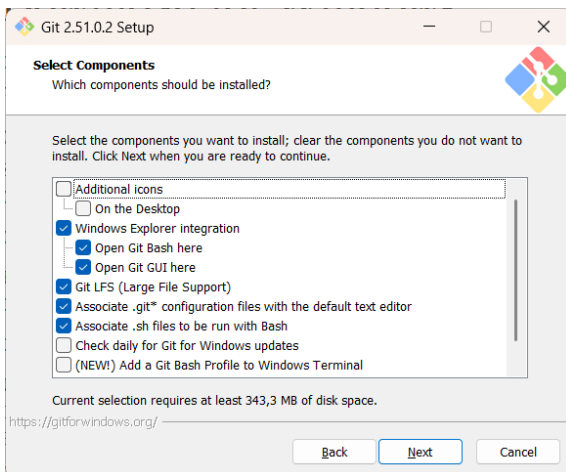
1



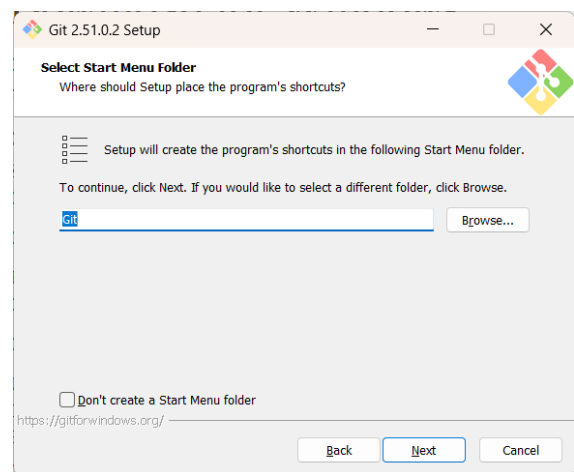
2



3

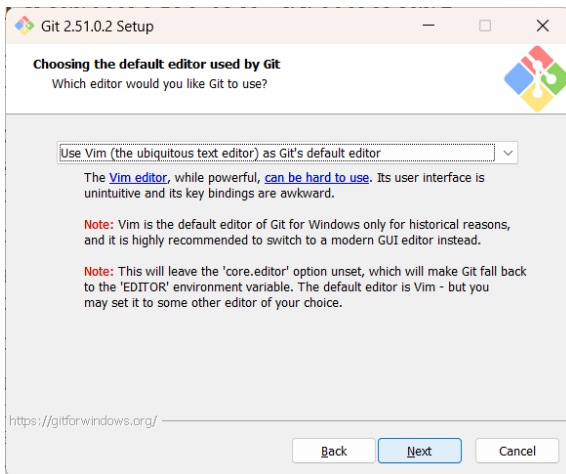


4

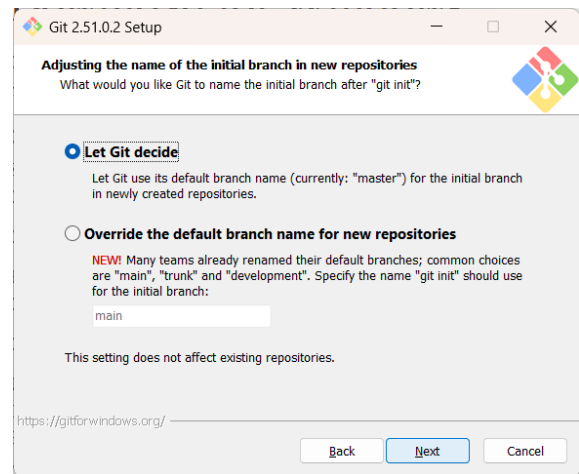


5

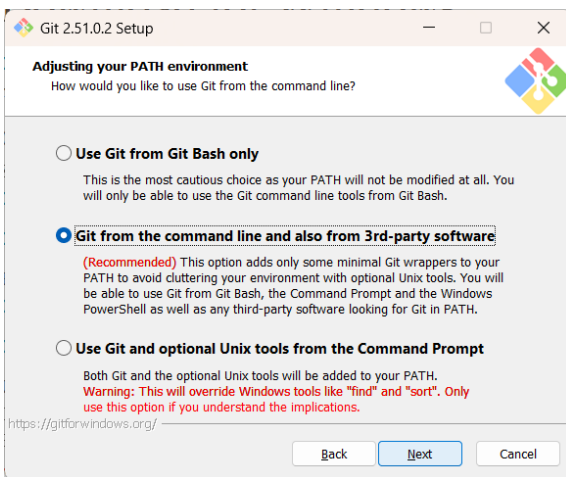
6



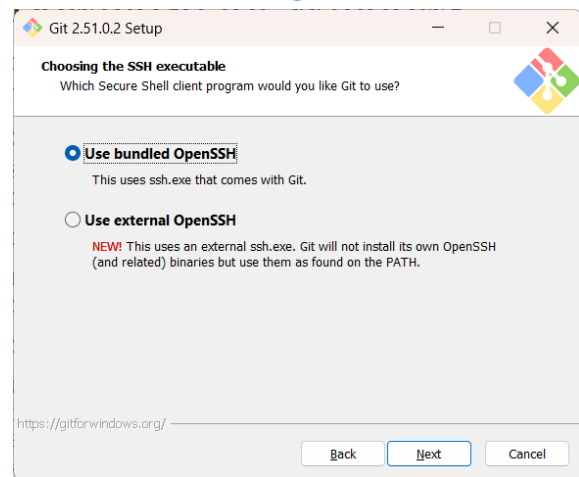
7



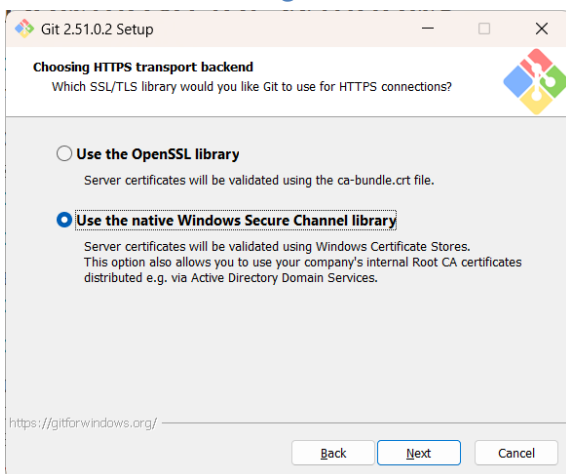
8



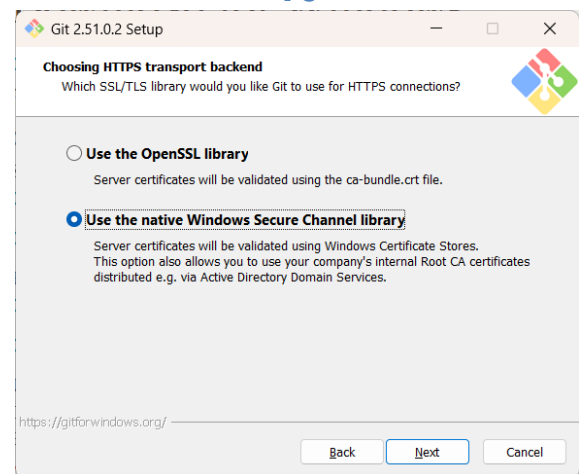
9



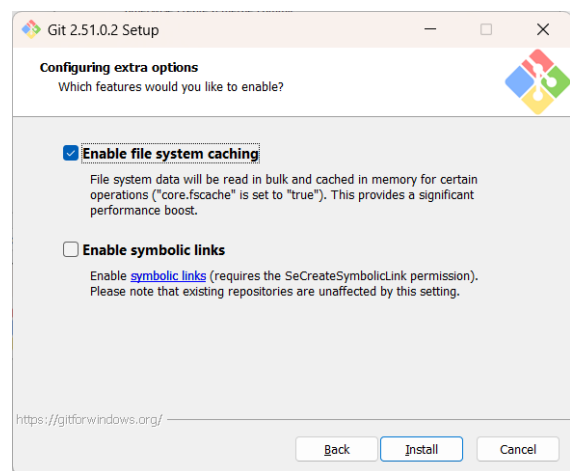
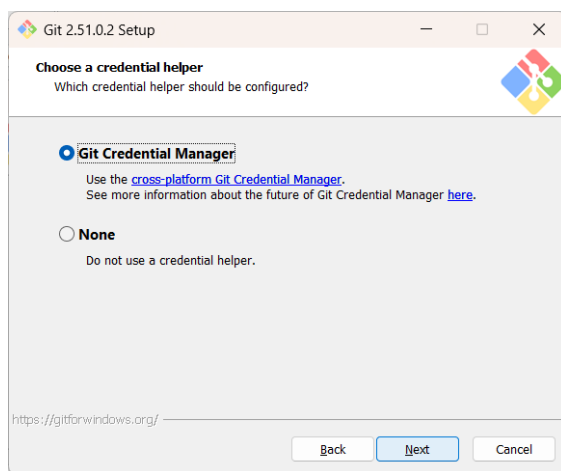
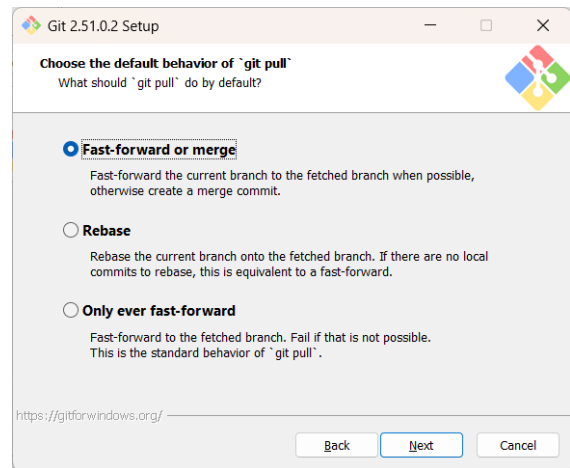
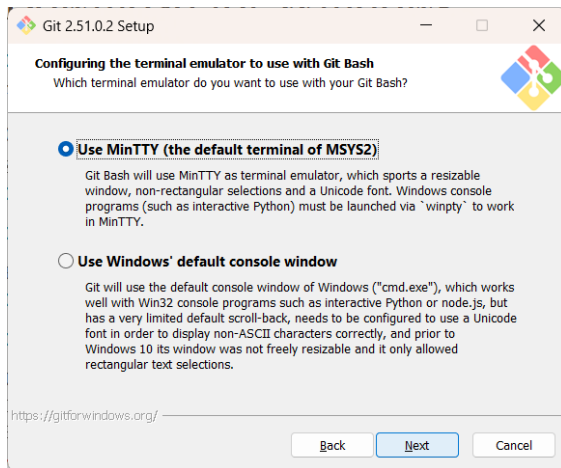
10



11

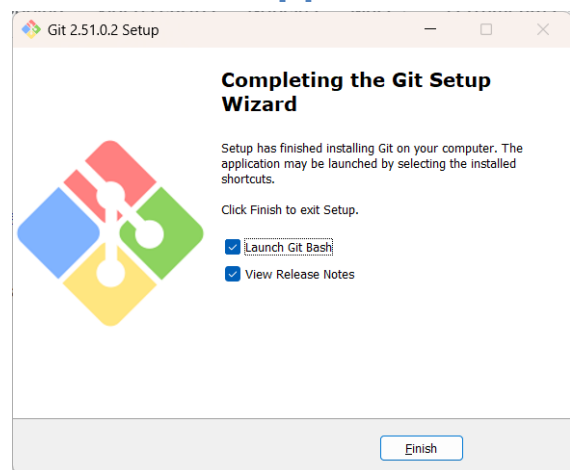
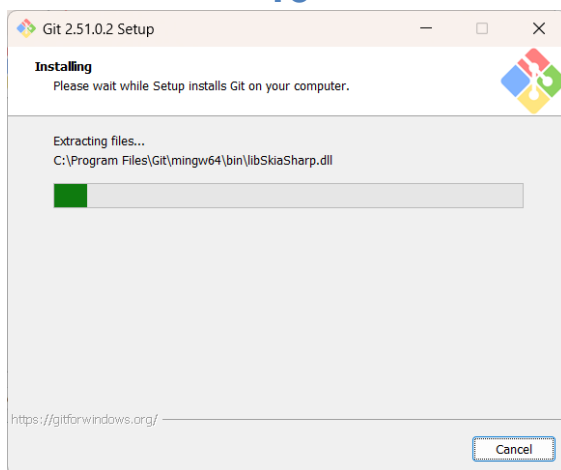


12

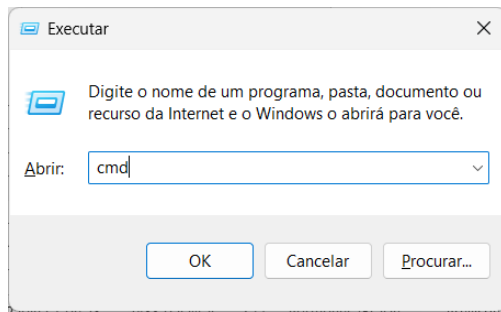


13

14



Após a instalação, abra o Prompt de Comando (Windows) ou Terminal (Mac/Linux).



Para executar o Prompt no Windows, acesse o menu Windows – opção Executar. Digite o comando: cmd

Para verificar a instalação digite o seguinte comando:

terminal

git --version

```
Microsoft Windows [versão 10.0.26100.6725]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\Paulo>git --version
git version 2.51.0.windows.2

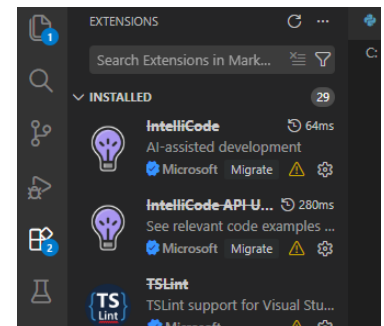
C:\Users\Paulo>
```

Agora que já temos os softwares necessários instalados vamos iniciar a configuração dos mesmos. Vamos iniciar a configuração do VSCode.

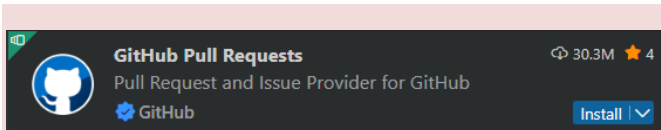
Adicionando extensões no VSCode

O Visual Studio Code permite adicionar funcionalidades extras por meio de extensões, que funcionam como complementos capazes de ampliar os recursos do editor. Existem milhares de extensões disponíveis, muitas delas voltadas para facilitar o trabalho com Git, GitHub e documentação de projetos.

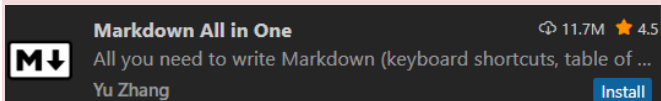
Para instalar uma extensão, abra o menu lateral esquerdo do VS Code e clique no ícone de Extensões, representado por um pequeno quadrado formado por quatro blocos. Em seguida, utilize o campo de busca para localizar e instalar as extensões desejadas.



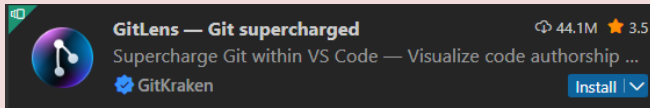
Algumas extensões importantes:



GitHub Pull Requests and Issues oferece integração oficial com o GitHub, permitindo visualizar e gerenciar pull requests diretamente dentro do editor.



Markdown All in One facilita a criação e edição de arquivos no formato Markdown, muito utilizados em arquivos README para documentar projetos.



GitLens permite visualizar o histórico detalhado das alterações realizadas nos arquivos, mostrando informações sobre commits, autores e modificações ao longo do tempo.

Embora nem todas sejam obrigatórias, essas extensões podem tornar o ambiente de desenvolvimento mais completo e ajudar no acompanhamento das mudanças realizadas nos projetos.

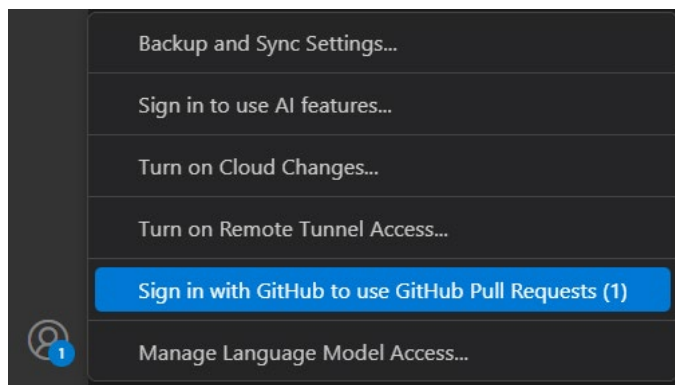
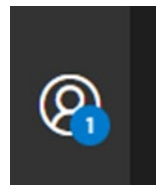


Sempre mantenha o Git e o VS Code atualizados. Versões recentes corrigem erros e trazem melhorias importantes, especialmente na integração com o GitHub.

Login no GitHub no VSCode

Para utilizar os recursos do GitHub diretamente no Visual Studio Code, é necessário fazer login com a sua conta na plataforma. Essa integração permite conectar o editor ao seu perfil no GitHub, facilitando o gerenciamento de repositórios, o envio de arquivos e o acompanhamento de projetos sem precisar sair do ambiente de desenvolvimento. O próprio VS Code oferece integração nativa com o GitHub, o que torna esse processo simples e rápido. Vamos visualizar o passo a passo para realizar esse login e preparar o editor para trabalhar de forma conectada com seus projetos online.

1. Com o VS Code aberto, clique no ícone de conta (canto inferior esquerdo da tela).
2. Selecione a opção Sign in with GitHub.



3. O navegador será aberto para você autorizar o VS Code a acessar sua conta do GitHub.



Sign in to GitHub
to continue to Visual Studio Code

Username or email address

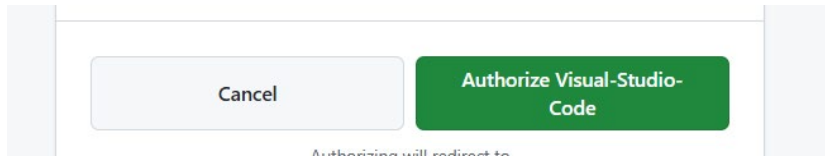
Password [Forgot password?](#)

[Sign in](#)

or

 Continue with Google

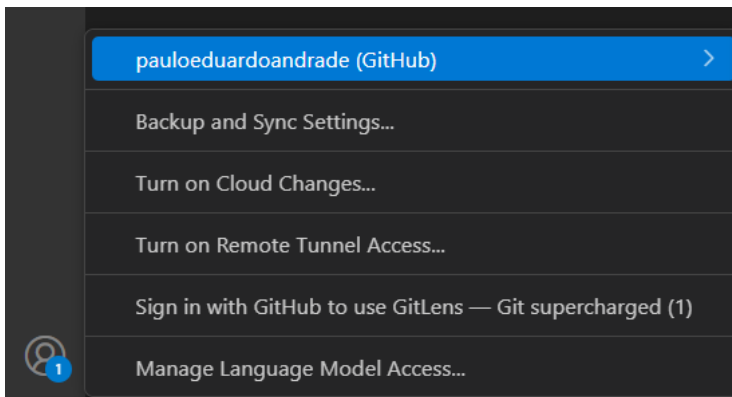
4. Confirme o login e retorne ao VSCode.



Cancel [Authorize Visual-Studio-Code](#)

Authorizing will redirect to:

5. Verifique no canto inferior esquerdo: agora deve aparecer o seu usuário conectado.



Dica - Autenticação no primeiro login

Ao fazer login pela primeira vez no GitHub pelo VS Code, pode ser que o sistema peça uma verificação em duas etapas. Isso acontece para garantir a segurança da sua conta.

- Normalmente, o GitHub envia um código para o seu e-mail cadastrado.
- Basta abrir a mensagem, copiar o código e colar na tela de login.
- Depois disso, o acesso será concluído normalmente.

Não se assuste se esse passo aparecer: é apenas uma medida extra de segurança para proteger seus projetos.

Criar ou abrir um projeto no VS Code

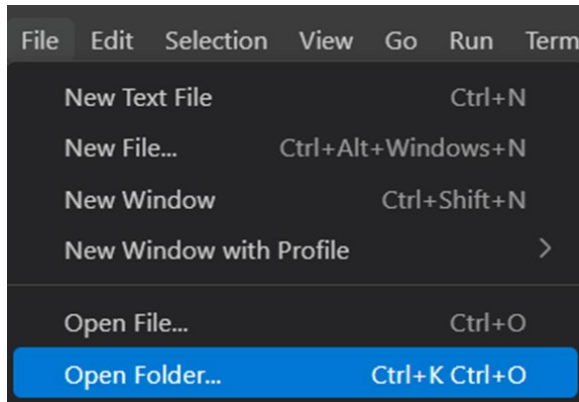
Para organizar melhor nosso aprendizado, vamos criar uma pasta principal chamada projetos. Dentro dela, cada novo trabalho ficará em uma subpasta, como se fosse uma gaveta exclusiva para cada projeto.

Como criar essa organização?

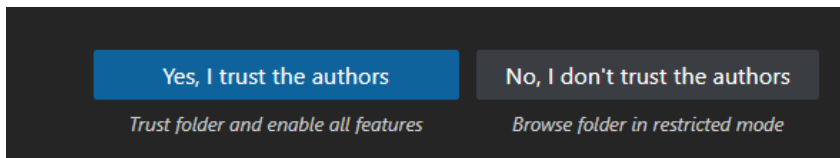
1. Crie uma pasta principal chamada Projetos (pode ser na Área de Trabalho ou Documentos) e dentro desta pasta um arquivo "primeiro-projeto".



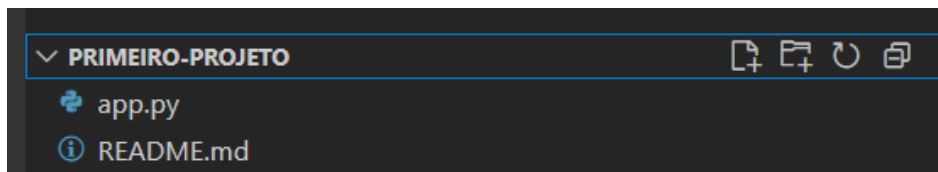
2. Abra o Visual Studio Code.
 - Clique em File → Open Folder ou Arquivo → Abrir Pasta.
 - Selecione a pasta primeiro-projeto (a pasta principal).



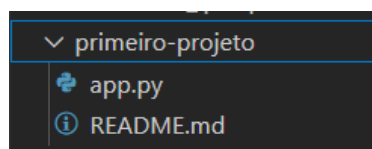
3. Achei a opção Yes, I trust the authors ou Sim, confio nos autores.



4. Clique na opção criar arquivo , ela aparecerá assim que passar o mouse na pasta primeiro-projeto.



5. Crie dois arquivos:
 - app.py
 - README.md



Para nosso primeiro exemplo vamos utilizar o arquivo app.py, vamos criar um pequeno programa de média que segue o ciclo entrada, processamento e saída.

Veja o código a seguir:

.python

```
# Programa de cálculo de média de notas
# Autor: Seu Nome

# Entrada
nome = input("Digite o nome do aluno: ")
nota1 = float(input("Digite a primeira nota: "))
nota2 = float(input("Digite a segunda nota: "))

# Processamento
media = (nota1 + nota2) / 2

# Saída
print(f"\nAluno: {nome}")
print(f"Média: {media:.2f}")

if media >= 6:
    print("Situação: Aprovado")
else:
    print("Situação: Reprovado")
```

No arquivo README.md, vamos criar como uma capa ou manual do seu projeto, permitindo que qualquer pessoa entenda rapidamente o que foi feito, por que foi feito e como utilizar. Veja a seguir o código do README:

.markdown

📄 Primeiro Projeto - Cálculo de Média

Este projeto em Python foi criado para praticar o ciclo ****Entrada → Processamento → Saída****.

O programa solicita o ****nome do aluno**** e duas ****notas****, calcula a média e mostra se o aluno foi aprovado ou reprovado.

O papel do README em cada projeto

Sempre que você criar um projeto no GitHub, é fundamental incluir um arquivo chamado README.md. Ele funciona como uma capa ou manual do seu projeto, permitindo que qualquer pessoa entenda rapidamente o que foi feito, por que foi feito e como utilizar.

Sem um README, o repositório pode parecer incompleto ou confuso. Já com um README bem escrito, o projeto ganha clareza, profissionalismo e se torna mais útil para quem acessa.

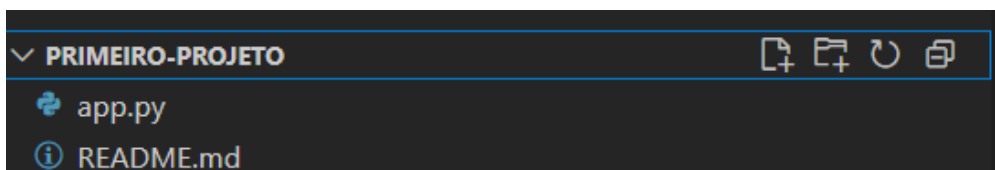
O que incluir em um bom README para projetos do seu portfólio?

- Título e descrição: qual é o projeto e o que ele faz.
- Objetivo: o que você pretendeu treinar ou resolver com esse código.
- Instruções de uso: como executar o programa no computador.
- Exemplos: pequenos trechos de saída, prints ou explicações que mostrem o resultado esperado.
- Tecnologias usadas: indique a linguagem ou ferramentas utilizadas (ex.: Python, HTML, etc.).

Inicializar o repositório Git no VS Code

Agora que criamos nosso primeiro projeto, precisamos transformá-lo em um repositório Git local. Isso significa que o Git passará a controlar as versões dos arquivos, registrando todas as alterações feitas ao longo do tempo.

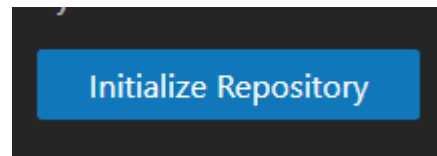
1. Com a pasta primeiro-projeto aberta no VS Code.



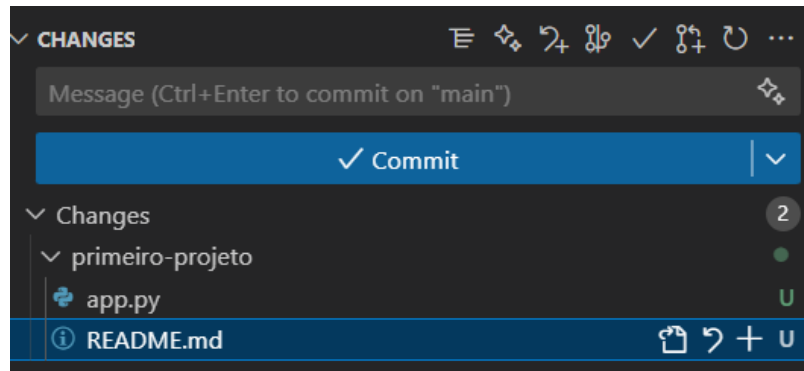
2. No menu lateral esquerdo, clique no ícone de Source Control (Controle de Código-Fonte, representado por um símbolo de ramificação).



3. Clique em Initialize Repository (Inicializar Repositório).



Isso criará automaticamente uma pasta oculta chamada `.git`, onde o Git guarda o histórico de versões.

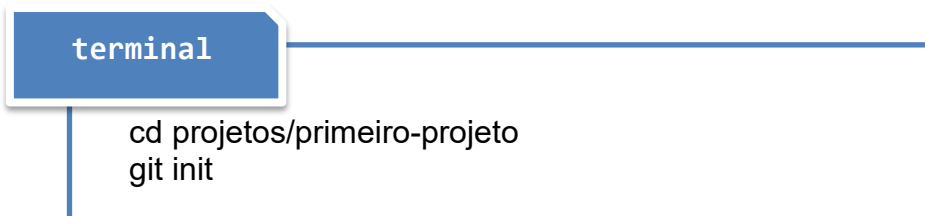


O que acontece agora?

- Seus arquivos do projeto (`app.py` e `README.md`) passam a ser monitorados pelo Git.
- Eles aparecem no painel de Source Control como Untracked (não rastreados).
- A partir daqui, sempre que você modificar, adicionar ou remover arquivos, o Git registrará as mudanças.

Alternativa via terminal

Uma outra opção é inicializar o repositório pelo terminal, para isso basta digitar o código:



O código acima cria a pasta `.git` na raiz do projeto.

Primeiro Commit

Um commit pode ser entendido como uma espécie de “foto” do projeto em um determinado momento. Sempre que você realiza um commit, o Git registra quais arquivos foram alterados, quem fez a alteração, quando ela foi realizada e qual foi o motivo da mudança, descrito por meio da mensagem de commit.

De acordo com Chacon e Straub (2014), um commit representa o registro das alterações realizadas no repositório, acompanhado de uma mensagem que descreve essas mudanças e cria um ponto na linha do tempo do projeto. Esse registro permite acompanhar a evolução do código e recuperar versões anteriores sempre que necessário.

Ao longo do desenvolvimento de um projeto, vários commits vão sendo registrados e formam o histórico do projeto. Esse histórico permite entender quais mudanças foram feitas ao longo do tempo e até mesmo retornar para versões anteriores caso algo dê errado.

Por esse motivo, o commit é considerado uma unidade básica de trabalho no Git. Cada commit deve representar uma alteração clara e organizada no projeto. Uma boa prática é registrar apenas uma ideia ou modificação por commit, facilitando a compreensão das mudanças realizadas.

Exemplos mensagens de commit:

Prefixo	Português	English	Uso principal
feat	funcionalidade	feature	Adicionar uma nova funcionalidade ao código.
fix	correção	fix	Corrigir um bug ou erro existente.
docs	documentação	docs	Alterações na documentação (README, comentários, etc.)
style	estilo	style	Ajustes de formatação/estilo (espaços, indentação, sem afetar lógica).
refactor	refatoração	refactor	Reescrever código sem mudar o comportamento (melhorias internas).
test	testes	test	Adicionar ou alterar testes automatizados.
chore	tarefa rotineira	chore	Manutenção geral (build, configs, dependências, sem impacto direto no código).
perf	desempenho	perf	Alterações que melhoram a performance.
ci	integração contínua	ci	Alterações em pipelines/configuração de integração contínua.
build	build/compilação	build	Mudanças em dependências, scripts de build ou ferramentas externas.

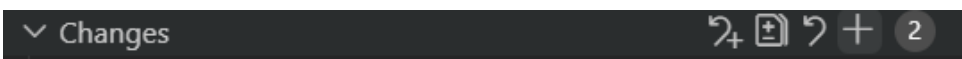
Alguns exemplos práticos em português:

- feat: adiciona cálculo de média no app.py.
- fix: corrige divisão por zero.
- docs: atualiza instruções no README.

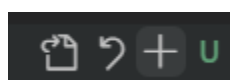
Passo a passo no VSCode

{ Para iniciar o commit o repositório precisa já está inicializado e arquivos (app.py e README.md) criados. }

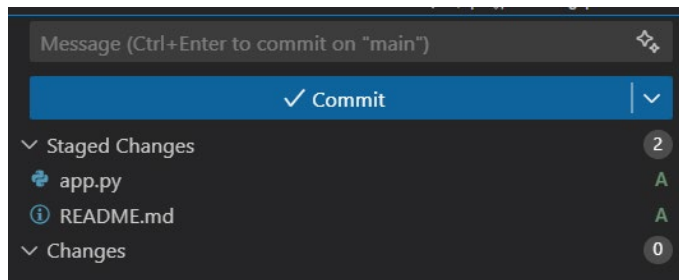
1. Abra a pasta do projeto no VS Code. Exemplo: projetos/primeiro-projeto.
2. Vá em Source Control (ícone de ramificação à esquerda)
Seus arquivos aparecerão em Changes (não rastreados/modificados).



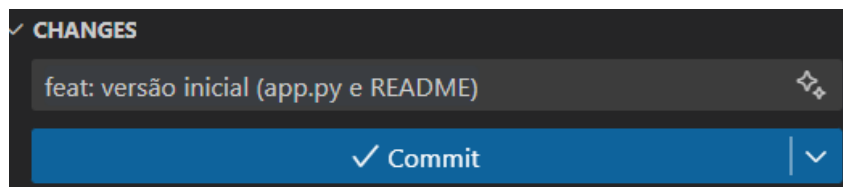
3. Stage (marcar para commit)
Clique no + ao lado de cada arquivo (ou em *Stage All*).



Os itens irão para Staged Changes.

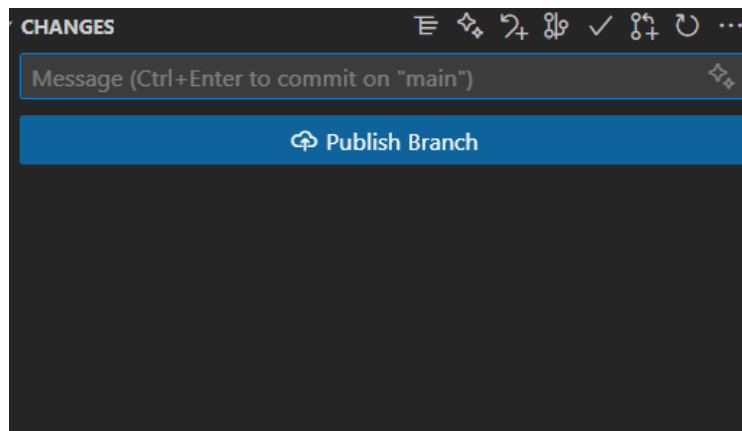


4. No campo de mensagem, digite algo como: feat: versão inicial (app.py e README).
5. Para confirmar o commit, clique em Commit (ou use Ctrl+Enter).



Se aparecer um aviso, confirme.

6. Verifique o resultado:



A lista de mudanças ficará vazia (ou só com arquivos ainda não salvos).

Você também pode realizar a alteração diretamente no terminal.

terminal

```
git status
git add .
git commit -m "feat: versão inicial (app.py e README)"
```

Entendendo os estados:

- **Untracked:** arquivo novo, ainda não monitorado.
- **Modified:** arquivo já conhecido pelo Git, mas alterado.
- **Staged:** pronto para entrar no próximo commit (após o add).
- **Committed:** mudança registrada no histórico

Erros comuns

- **Commit não habilita:** provavelmente nada está em Staged Changes - faça o Stage.
- **Nada aparece no Source Control:** repositório não inicializado - volte ao Item inicializar o git.
- **Commit vazio:** sem alterações novas - edite um arquivo ou verifique o Stage.

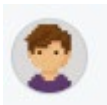
Publicar o repositório no GitHub

Agora que já criamos o projeto, inicializamos o repositório e fizemos o primeiro commit, chegou a hora de publicar no GitHub. Isso significa enviar o histórico do seu projeto local para a nuvem, criando um repositório remoto. Assim, além de manter uma cópia de segurança, você poderá mostrar seus projetos para professores, colegas e até recrutadores.

1. Abra o projeto no VS Code (ex.: projetos/primeiro-projeto).
2. Vá até o painel Source Control (ramificação à esquerda).

**Antes de publicar usar o e-mail “noreply” do GitHub**

No GitHub (web): clique em Settings, Emails.



1

Settings

2

Emails

3

Copie o seu e-mail noreply (formato: ID+seuusuario@users.noreply.github.com).

Keep my email addresses privateOn ☒

We'll remove your public profile email and use 146391254+pauloeduardoandrade@users.noreply.github.com when performing web-based Git operations (e.g. edits and merges) and sending email on your behalf. If you want command line Git operations to use your private email you must [set your email in Git](#).

Previously authored commits associated with a public email will remain public.

1.

Confirme que a opção “Keep my email addresses private” está ligada.

**Por que usar o e-mail “noreply” do GitHub:**

O Git registra o nome e o e-mail do autor em cada commit.

Se você usar seu e-mail pessoal, ele ficará visível publicamente no histórico do repositório.

Usar o e-mail “noreply” do GitHub (ex.: 12345678+usuario@users.noreply.github.com) mantém sua privacidade e protege seus dados pessoais, sem alterar o funcionamento dos commits.

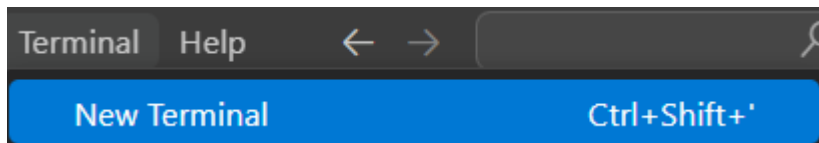
Além disso:

- *Evita o erro GH007 (“Your push would publish a private email address”).*

```
> git push -u origin main
remote: error: GH007: Your push would publish a private email address.
remote: You can make your email public or disable this protection by visiting:
remote: https://github.com/settings/emails
```

- *Permite publicar sem expor seu endereço real.*
- *É a configuração recomendada para estudantes e iniciantes.*

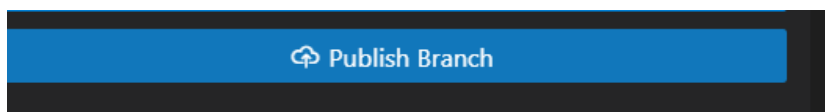
No seu projeto (terminal dentro da pasta do repo):



terminal

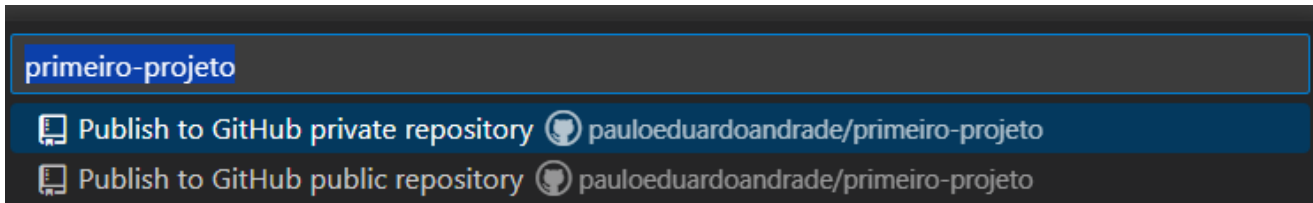
```
git config --global user.email "ID+seuusuario@users.noreply.github.com"
git config --global user.name "Seu Nome"
```

Clique em Publish to GitHub (Publicar no GitHub).

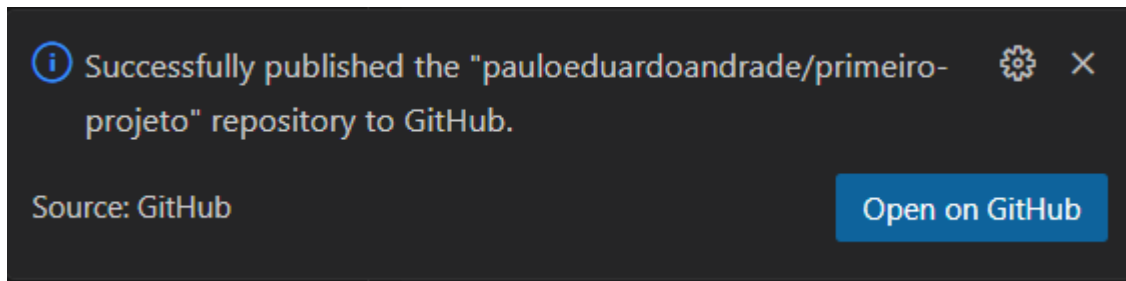


Escolha:

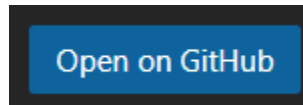
- Public: qualquer pessoa pode ver o repositório.
- Private: apenas você e quem você autorizar.



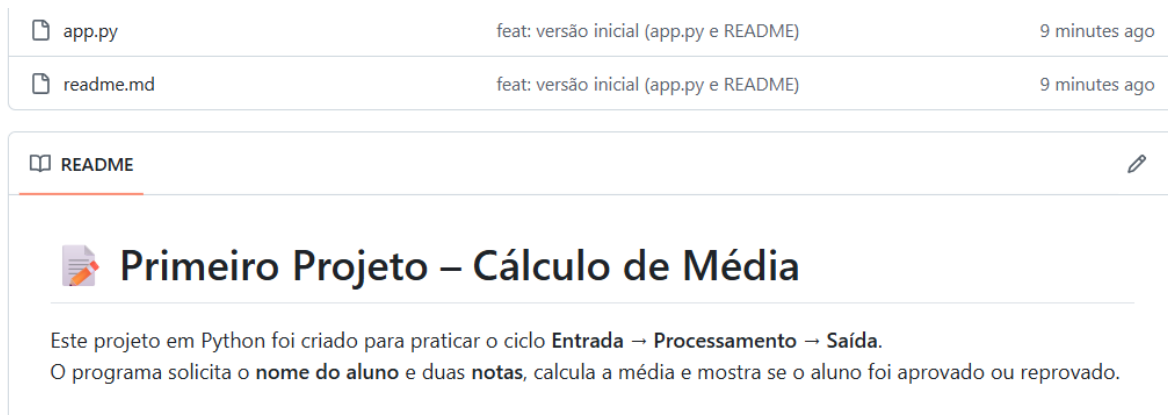
O VS Code vai criar o repositório remoto e enviar os arquivos automaticamente.



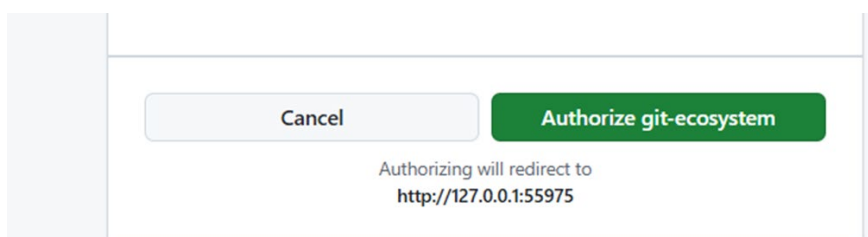
Ao final, aparecerá um link para abrir no navegador, clique e confira seu repositório já no GitHub.



Após o click você verá seu repositório no GitHub.



Observação: Talvez apareça para autenticar o git-ecosystem.



Botão Publish Branch:

Esse botão aparece quando:

- Você já fez pelo menos um commit local,
- E o repositório ainda não está conectado ao GitHub (não tem remoto).

Ao clicar em “Publish Branch”, o VS Code:

1. Cria automaticamente um repositório remoto no GitHub,
2. Faz o primeiro push,
3. E sincroniza o branch local (main) com o remoto.

Passo a passo alternativo (terminal):

Se preferir pode criar manualmente pelo site do GitHub, seguindo os passos:

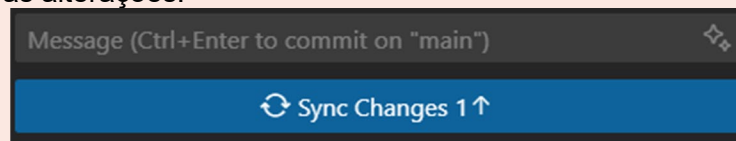
1. No GitHub, clique em New Repository.
2. Dê o mesmo nome da sua pasta local (primeiro-projeto).
3. Não adicione README, para evitar conflito.
4. No terminal, dentro da pasta do projeto, digite:

terminal

```
git remote add origin https://github.com/SEU-USUARIO/primeiro-projeto.git
git branch -M main
git push -u origin main
```

Importante:

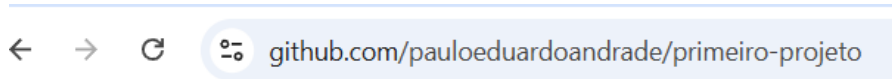
- A mensagem de commit já foi escrita antes (quando você fez o commit local).
- “Publish Branch” apenas envia esse commit para o GitHub — não cria um novo.
- Depois que publicar, o botão desaparece e passa a mostrar “Synchronize Changes” ou “Push/Pull” conforme houver novas alterações.

**Compartilhando seu projeto no GitHub**

Depois de publicar seu repositório no GitHub, é hora de compartilhar o link com outras pessoas. Esse link permite que professores, colegas e desenvolvedores visualizem o seu trabalho, entendam sua forma de programar e até colaborem com sugestões de melhorias. Ter um repositório bem configurado (com foto, bio, README, commits, badges e organização) ajuda a construir um portfólio profissional e mostra sua evolução como desenvolvedor.

1. Acesse seu repositório no GitHub

- Vá até <https://github.com> e clique no projeto que deseja compartilhar (exemplo: primeiro-projeto).
- O endereço do repositório aparecerá na barra de navegação, semelhante a:
 - <https://github.com/seuusuario/primeiro-projeto>



2. Copie o link e Compartilhe com outros desenvolvedores

- Envie o link em grupos de estudo, portfólios online, LinkedIn, currículo digital ou fóruns de programação.
- Assim, outras pessoas poderão conhecer seu código, deixar comentários, sugerir melhorias ou até contribuir com novas ideias.

**Dica:**

O GitHub é uma vitrine do seu trabalho. Compartilhar seus repositórios mostra seu progresso e pode abrir portas para colaborações, estágios e oportunidades profissionais.

Assista a vídeo aula da professora Tatiane Assis:



Disponível em: <https://www.youtube.com/watch?v=S726vNUYIMc>

Acessado em 13/03/2026.